

How main.py Works

DAX AI Financial Reporter

April 2026

Overview

`main.py` is the single entry point for the DAX AI Financial Reporter. It runs a five-step pipeline that fetches live market data, analyses it, asks Claude to write a narrative commentary, and saves a finished report to disk.

`main()`

- | | | |
|---|------------------------------|---------------------------------------|
| 1. <code>get_dax_tickers()</code> | <code>pytickersymbols</code> | → list of {name, ticker, industries} |
| 2. <code>fetch_ohlc()</code> | <code>yfinance</code> | → (close DataFrame, volume DataFrame) |
| 3. <code>analyze()</code> | <code>pandas</code> | → metrics dict |
| 4. <code>generate_ai_commentary()</code> | Claude API | → commentary string |
| 5. <code>render_report() + save_report()</code> | | → .md or .html file on disk |
-

Step 1 — `get_dax_tickers()`

What it does: Returns a list of dicts, one per DAX 40 constituent, containing the company name, its Yahoo Finance ticker symbol (e.g. `SAP.DE`), and a list of industry labels.

How it works: The `PyTickerSymbols` library ships an offline database of index members. The function iterates over every stock in the "DAX" index and picks the ticker listed on **XETRA** (the primary German exchange). If no XETRA entry is found, it falls back to any ticker ending in `.DE`. Stocks with no usable ticker are skipped.

Step 2 — `fetch_ohlc(tickers)`

What it does: Downloads Open/High/Low/Close/Volume data for every ticker and returns two DataFrames — `close` and `volume` — each covering the last **5 trading days**.

How it works: `yfinance.download()` is called with `period="15d"` to ensure enough raw rows even around weekends and public holidays. After dropping rows where all values are NaN, `.tail(5)` trims to the five most recent trading days. `auto_adjust=True` applies split and dividend adjustments automatically.

Step 3 — `analyze(stocks, close, volume)`

What it does: Computes all quantitative metrics and returns them in a single dictionary that the rest of the pipeline consumes.

Metrics produced:

Metric	How it is calculated
5-day return per stock	<code>(last_close - first_close) / first_close × 100</code>
Top 5 gainers / losers	<code>pandas Series.nlargest(5) / nsmallest(5)</code> on returns
Volume spike	<code>(last_day_volume - 5d_avg) / 5d_avg × 100</code>
Top 5 volume spikes	<code>nlargest(5)</code> on the spike series
Sector average return	Returns grouped by the first industry label, then averaged

The output dict also includes human-readable `date` and `period` strings used directly in the report heading.

Step 4 — `generate_ai_commentary(analysis)`

What it does: Calls the Anthropic Claude API and returns a 2–3 paragraph narrative commentary about the week’s market activity.

How it works:

1. `_build_data_summary()` serialises the metrics dict into a plain-text block (gainers, losers, volume spikes, sector performance) that Claude can read.
2. A `messages.create()` call is made to `claude-sonnet-4-6` with:
 - A **cached system prompt** (using `cache_control: ephemeral`) that establishes the analyst persona. Caching avoids re-processing this static text on repeated runs, reducing latency and cost.
 - A user message containing the data summary and a specific instruction to structure the response into three thematic paragraphs.
3. If `ANTHROPIC_API_KEY` is not set, the function returns a placeholder string instead of raising an error, so the rest of the pipeline still produces a report.

Step 5 — `render_report()` and `save_report()`

What they do: Combine all metrics and the AI commentary into a formatted document, then write it to `output/reports/`.

`render_report()` builds a Markdown string with:

- A title and 5-day summary line.
- Four Markdown tables (gainers, losers, volume spikes, sector performance) constructed by the helper `_md_table()`.
- The Claude commentary block, labelled as AI-generated.

`save_report()` creates the `output/reports/` directory if it does not exist, then writes the file with a date-stamped name such as `dax_report_2026-04-24.md`.

If `REPORT_FORMAT=html` is set in `.env`, the Markdown string is first converted to HTML with the `markdown` library (using the `tables` extension), wrapped in a minimal HTML shell with inline CSS, and saved as a `.html` file instead.

Configuration via `.env`

Variable	Default	Effect
<code>ANTHROPIC_API_KEY</code>	<i>(none)</i>	Required for AI commentary
<code>REPORT_OUTPUT_DIR</code>	<code>output/reports</code>	Where reports are saved
<code>REPORT_FORMAT</code>	<code>markdown</code>	<code>markdown</code> → <code>.md</code> , <code>html</code> → <code>.html</code>

Running the Script

```
source .venv/bin/activate
python main.py
```

Expected console output:

```
DAX AI Financial Reporter
=====
1/5  Fetching DAX 40 tickers...
      40 stocks found
2/5  Downloading OHLCV data (last 5 trading days)...
      40 tickers × 5 days
3/5  Analyzing market data...
4/5  Generating AI commentary via Claude...
5/5  Rendering and saving report...

Done. Report saved to: output/reports/dax_report_2026-04-24.md
```